



EXPERIMENT -9

Student Name: Vinay Tiwari

Branch: AIT-CSE (AIML)

Semester: 5

Subject Name: Database Management System

UID: 24BAI70919

Section/Group: 24AIT_KRG-1A

Subject Code: 24CSH-298

Q1) An organization stores employee details in the employee_info table. Create a PostgreSQL stored procedure that accepts an employee's gender as an input parameter and calculates the total number of employees belonging to that gender. The calculated count should be returned through an OUT parameter and should also be displayed as a message using RAISE NOTICE.

For example, if Male is passed to the procedure, it should count all employees whose gender is Male.

Table: employee_info

Column Name	Data Type	Description
id	SERIAL	Unique employee ID
name	VARCHAR(50)	Employee name
gender	VARCHAR(10)	Gender of employee
salary	NUMERIC(10,2)	Employee salary
city	VARCHAR(50)	Employee city

SOFTWARE REQUIREMENTS:

Database Management System:

PostgreSQL Database

Database Administration Tool / Client Tool:

pgAdmin (for PostgreSQL)

EXPERIMENT -9

CODE SCREENSHOTS:

```
EXPERIMENT -9
CREATE OR REPLACE PROCEDURE SP_COUNT_GENDER(IN P_GENDER VARCHAR(10), OUT P_NUM INT)
LANGUAGE PLPGSQL
AS
$$
BEGIN
    --LOGIC
    SELECT COUNT(ID) INTO P_NUM FROM EMPLOYEE_INFO
    WHERE GENDER=P_GENDER;

    RAISE NOTICE 'THE COUNT OF EMPLOYEE HAVING GENDER AS % IS %', P_GENDER, P_NUM;
END;
$$

call SP_COUNT_GENDER('Male', 0)
```

OUTPUT SCREENSHOTS:

	p_num integer
1	6

LEARNING OUTCOMES:

After completing this experiment, I was able to:

- Understand the concept of **stored procedures** in PostgreSQL.
- Create a procedure using **IN and OUT parameters** to pass and return values.
- Use **COUNT()** with a **WHERE** clause to count employees of a given gender.
- Display results using **RAISE NOTICE** inside a PL/pgSQL block.
- Execute a procedure using the **CALL** statement and read the returned value.

CONCLUSION:

In conclusion, this task demonstrates how a stored procedure can accept input values, process data stored in a table and return a result through an OUT parameter. By counting employees based on the gender passed as input and displaying the result using RAISE NOTICE, we can build reusable and efficient database logic.

EXPERIMENT -9

Q2) An e-commerce company maintains product stock information in the products table and stores completed orders in the sales table. Create a PostgreSQL stored procedure that accepts a product name and required quantity as input parameters and processes the purchase only when sufficient stock is available.

The procedure should verify the available quantity of the requested product. If sufficient stock exists, it should retrieve the product code and price, calculate the total sale amount, insert a new order into the sales table, reduce the available stock, and increase the total quantity sold. If sufficient stock is not available, the procedure should not modify any data and should display an appropriate notification message.

Sale Price = Product Price × Quantity Ordered

Create a stored procedure named pr_buy_products that performs the following operations:

- Accept product_name and quantity as input parameters.
- Check whether the requested product exists and has sufficient stock.
- If sufficient stock is available:
 - Retrieve its product code and price.
 - Calculate the total sale amount.
 - Insert a new record into the sales table.
 - Reduce quantity_remaining by the purchased quantity.
 - Increase quantity_sold by the purchased quantity.
 - Display a success message.
- If sufficient stock is not available:
 - Do not insert or update any record.
 - Display an insufficient quantity message.

Table: products

Column Name	Data Type	Description
product_code	VARCHAR(10)	Unique product code
product_name	VARCHAR(100)	Product name
price	NUMERIC(10,2)	Price of one unit
quantity_remaining	INT	Current available stock
quantity_sold	INT	Total quantity sold

EXPERIMENT -9

Table: sales

Column Name	Data Type	Description
order_id	SERIAL	Unique order ID
order_date	DATE	Date on which order was placed
product_code	VARCHAR(10)	Product purchased
quantity_ordered	INT	Quantity purchased
sale_price	NUMERIC(10,2)	Total order amount

SOFTWARE REQUIREMENTS:

Database Management System:

PostgreSQL Database

Database Administration Tool / Client Tool:

pgAdmin (for PostgreSQL)

EXPERIMENT -9

CODE SCREENSHOTS:

```
CREATE OR REPLACE PROCEDURE pr_buy_products(  
    IN p_product_name VARCHAR,  
    IN p_quantity INT  
)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_product_code VARCHAR(20);  
    v_price FLOAT;  
    v_count INT;  
BEGIN  
  
    -- Step 1: Check if product exists and has enough quantity  
    SELECT COUNT(*)  
    INTO v_count  
    FROM products  
    WHERE product_name = p_product_name  
    AND quantity_remaining >= p_quantity;  
  
    -- Step 2: If sufficient stock  
    IF v_count > 0 THEN  
  
        -- Fetch product code and price  
        SELECT product_code, price  
        INTO v_product_code, v_price  
        FROM products  
        WHERE product_name = p_product_name;  
  
        -- Insert a new record into the sales table  
        INSERT INTO sales (order_date, product_code, quantity_ordered, sale_price)  
        VALUES (CURRENT_DATE, v_product_code, p_quantity, (v_price * p_quantity));  
  
        -- Update stock details  
        UPDATE products  
        SET quantity_remaining = quantity_remaining - p_quantity,  
            quantity_sold = quantity_sold + p_quantity  
        WHERE product_code = v_product_code;  
  
        -- Confirmation message  
        RAISE NOTICE 'PRODUCT SOLD...! Order placed successfully for % unit(s) of %.', p_quantity, p_product_name;  
  
    ELSE  
        -- Step 3: If stock is insufficient  
        RAISE NOTICE 'INSUFFICIENT QUANTITY...! Order cannot be processed for % unit(s) of %.', p_quantity, p_product_name;  
    END IF;  
END IF;
```

EXPERIMENT -9

```
CALL pr_buy_products ('MacBook Pro 14"', 1);
```

OUTPUT SCREENSHOTS:

```
NOTICE:  PRODUCT SOLD..! Order placed successfully for 1 unit(s) of MacBook Pro 14".  
CALL
```

LEARNING OUTCOMES:

After completing this experiment, I was able to:

- Create a stored procedure that performs **multiple operations** in a single call.
- Use **IF-ELSE conditional logic** to check stock availability before processing an order.
- Declare **local variables** and fetch values into them using **SELECT INTO**.
- Apply **INSERT** and **UPDATE** statements to record sales and maintain stock levels.
- Calculate the sale amount using **Product Price × Quantity Ordered** and show messages with **RAISE NOTICE**.

CONCLUSION:

In conclusion, this task demonstrates how stored procedures can automate a complete business workflow inside the database. By verifying stock before a purchase, inserting the order into the sales table and updating the product quantities only when sufficient stock exists, the procedure keeps the data consistent and prevents invalid orders.